

Unit-1. Concepts of NoSQL- Mongo DB

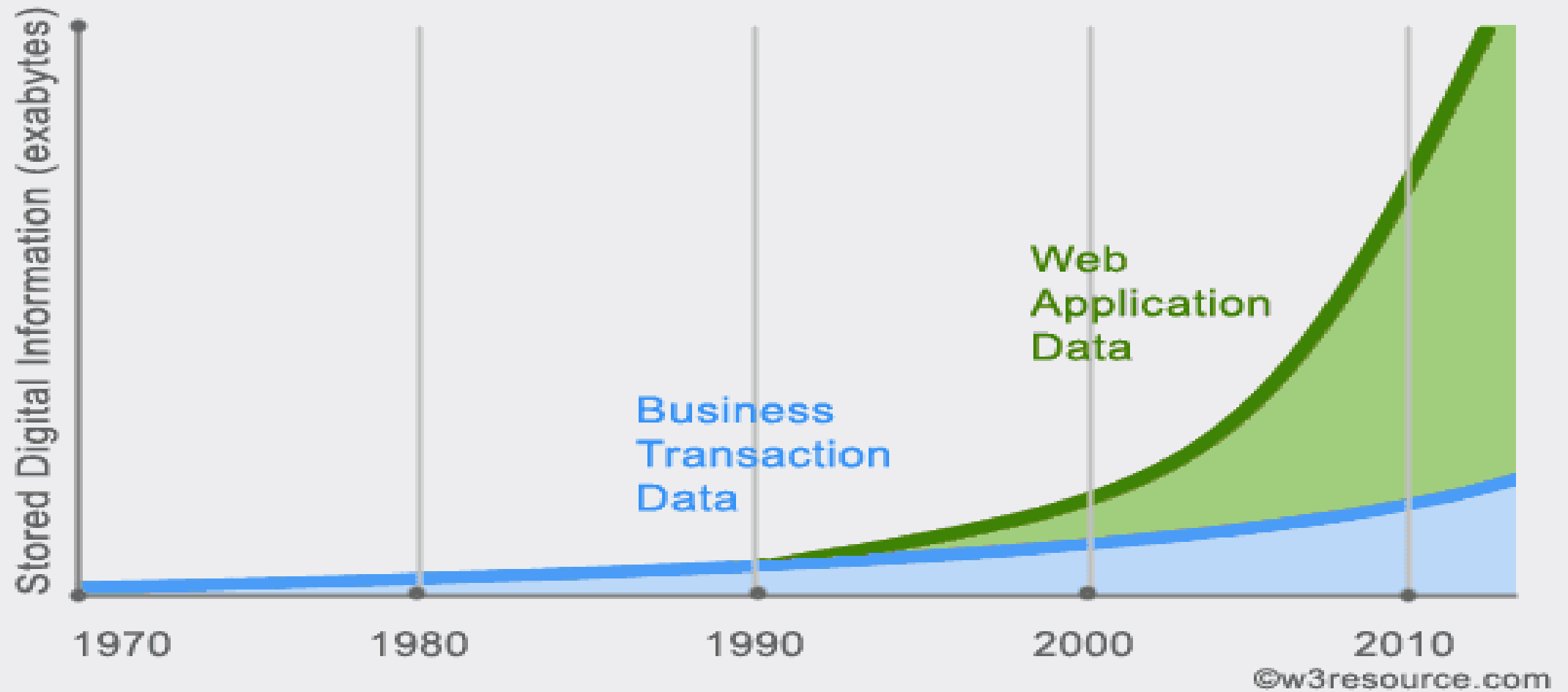
What is NoSQL?

- NoSQL is a non-relational database management systems, different from traditional relational database management systems in some significant ways.
- It is designed for distributed data stores where very large scale of data storing needs (for example Google or Facebook which collects terabits of data every day for their users).
- These type of data storing may not require fixed schema, avoid join operations and typically scale horizontally.

Why NoSQL?

- In today's time data is becoming easier to access and capture through third parties such as Facebook, Google+ and others. Personal user information, social graphs, geo location data, user-generated content and machine logging data are just a few examples where the data has been increasing exponentially.
- To avail the above service properly, it is required to process huge amount of data. Which SQL databases were never designed. The evolution of NoSql databases is to handle these huge data properly.

Web Applications Driving Data Growth



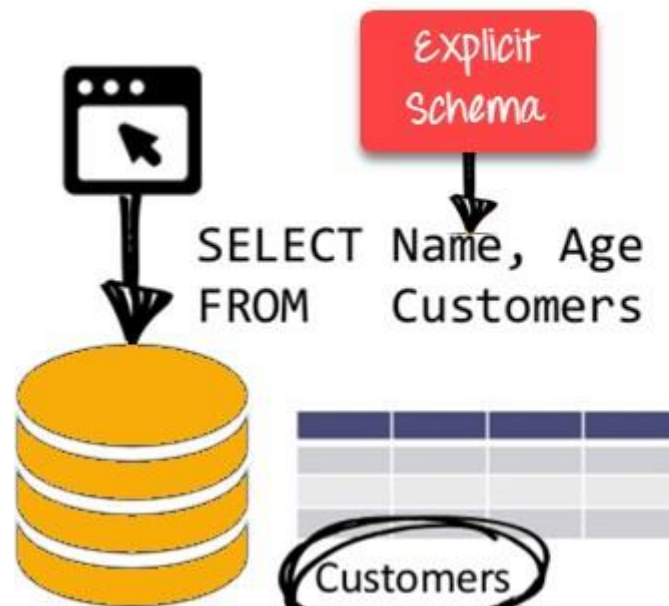
Features of NoSQL

- Flexible schemas
- Horizontal scaling
- Fast queries due to the data model
- Ease of use for developers

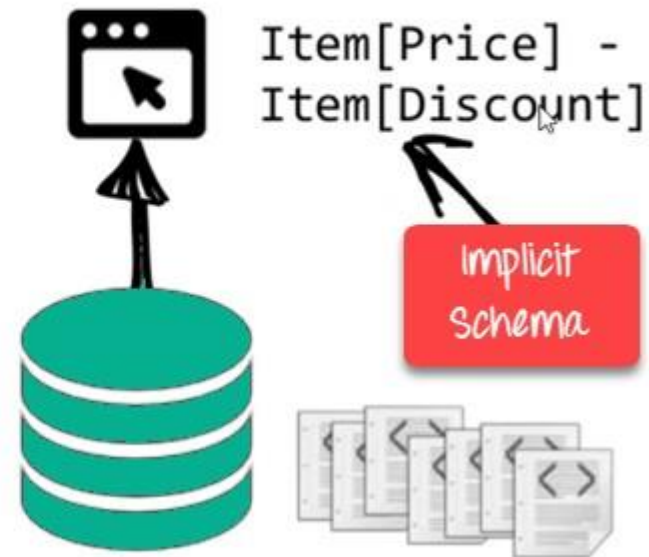
1. Flexible schemas

- NoSQL databases are either schema-free or have relaxed schemas
- Do not require any sort of definition of the schema of the data
- Offers heterogeneous structures of data in the same domain

RDBMS:



NoSQL DB:



2. Horizontal scaling

- Multiple NoSQL databases can be executed in a distributed fashion
- Offers auto-scaling and fail-over capabilities
- Often ACID concept can be sacrificed for scalability and throughput
- Mostly no synchronous replication between distributed nodes
Asynchronous Multi-Master Replication, peer-to-peer, HDFS Replication
- Only providing eventual consistency
- Shared Nothing Architecture. This enables less coordination and higher distribution.

3. Fast queries due to the data model

- NoSQL databases never follow the relational model
- Never provide tables with flat fixed-column records
- Work with self-contained aggregates or BLOBs
- Doesn't require object-relational mapping and data normalization
- No complex features like query languages, query planners, referential integrity joins, ACID

4. Ease of use for developers

- Offers easy to use interfaces for storage and querying data provided
- APIs allow low-level data manipulation & selection methods
- Text-based protocols mostly used with HTTP REST with JSON
- Mostly used no standard based NoSQL query language
- Web-enabled databases running as internet-facing services

NoSQL Advantages & Disadvantages

- **Advantages:**

- High scalability
- Distributed Computing
- Lower cost
- Schema flexibility, semi-structure data
- No complicated Relationships

- **Disadvantages**

- No standardization
- Limited query capabilities (so far)
- Eventual consistent is not intuitive to program for

NoSQL

Stands for Not Only SQL

- No declarative query language
- No predefined schema
- Key-Value pair storage, Column Store, Document Store, Graph databases
- Eventual consistency rather ACID property
- Unstructured data
- CAP Theorem
- Prioritizes high performance, high availability and scalability
- BASE Transaction

There is a large number of companies using NoSQL. To name a few :

- Google
- Facebook
- Mozilla
- Adobe
- Foursquare
- LinkedIn
- Digg
- McGraw-Hill Education
- Vermont Public Radio

MongoDB Datatypes

Data Types	Description
String	String is the most commonly used datatype. It is used to store data. A string must be UTF 8 valid in mongodb.
Integer	Integer is used to store the numeric value. It can be 32 bit or 64 bit depending on the server you are using.
Boolean	This datatype is used to store boolean values. It just shows YES/NO values.
Double	Double datatype stores floating point values.
Min/Max Keys	This datatype compare a value against the lowest and highest bson elements.
Arrays	This datatype is used to store a list or multiple values into a single key.
Object	Object datatype is used for embedded documents.
Null	It is used to store null values.
Symbol	It is generally used for languages that use a specific type.
Date	This datatype stores the current date or time in unix time format. It makes you possible to specify your own date time by creating object of date and pass the value of date, month, year into it.

Commands

Commands	Descriptions
show dbs	Print a list of all databases on the server.
use <db>	Switch current database to <db>.
show collections	Print a list of all collections for current database.
db.collection.find()	Find all documents in the collection
db.collection.findOne(<query>)	Find and return a single document. Returns null if not found.
db.collection.insertOne()	Insert a new document into the collection.
db.collection.insertMany()	Insert multiple new documents into the collection.
db.collection.updateOne()	Update a single existing document in the collection.
db.collection.updateMany()	Update multiple existing documents in the collection.
db.collection.deleteOne()	Delete a single document from the collection.
db.collection.deleteMany()	Delete documents from the collection.
db.collection.drop()	Drops or removes completely the collection.
db.collection.find().sort(<sort order>)	Return results in the specified <sort order>.
db.collection.find().limit(<n>)	Limit result to <n> rows.

_id

- _id is a 12 bytes hexadecimal number which assures the uniqueness of every document. You can provide _id while inserting the document. If you didn't provide then MongoDB provide a unique id for every document.
- These 12 bytes
 - first 4 bytes for the current timestamp,
 - next 3 bytes for machine id,
 - next 2 bytes for process id of mongod server and
 - remaining 3 bytes are simple incremental value.

Query with where_(not in syllabus)

- `db.mycollection.find({"key":"value"})` where key = value
- `db.mycollection.find({"key": {$lt: value}})` where key < value
- E.g. `db.mycollection.find({'marks' : {$lt : 75}})`
- `db.mycollection.find({"key": {$gt: value}})` where key > value
- `db.mycollection.find({"key": {$gte: value}})` where key >=value
- `db.mycollection.find({"key": {$ne: value}})` where key != value
- `db.mycollection.find({"key": {$lte: value}})` where key <=value
- `db.users.find({"name": /m/})` or `db.users.find({"name": /. *m. * /})` where name like '%m%'

And in Mongo Db

- `db.mycollection.find({"key1":"value1", "key2":"value2" })` where key1 = value1 and key2=value2

Or in Mongo Db

CRUD Operation

Create Database

>use db_name

Create Collection

>db.createCollection("posts")

Insert Document [insertOne()]

```
>db.posts.insertOne({  
  title: "Post Title 1",  
  body: "Body of post.",  
  category: "News",  
  likes: 1,  
  tags: ["news", "events"],  
  date: Date() })
```

Insert Document [insertMany()]

```
>db.posts.insertMany([
  {
    title: "Post Title 2",
    body: "Body of post.",
    category: "Event",
    likes: 2,
    tags: ["news", "events"],
    date: Date()
  },
  {
    title: "Post Title 3",
    body: "Body of post.",
    category: "Technology",
    likes: 3,
    tags: ["news", "events"],
    date: Date() }
]);
```

Update Document [updateOne()]

```
>db.posts.updateOne( { title: "Post Title 1" }, { $set: { likes: 2 } } )
```

Update Document [updateMany()]

```
>db.posts.updateMany({}, { $inc: { likes: 1 } })
```

Delete Document [deleteOne()]

```
>db.posts.deleteOne({ title: "Post Title 5" })
```

Delete Document [deleteMany()]

```
>db.posts.deleteMany({ category: "Technology" })
```

Limit Number of Rows

```
>db.COLLECTION_NAME.find().limit(NUMBER)
```

MongoDB Sort Documents Results

```
>db.COLLECTION_NAME.find().sort({KEY:1})
```

```
>db.mycol.find({ },{"username":1,_id:0}).sort({"username":-1})
```

Multiple Keys:

```
>db.COLLECTION_NAME.find().sort({KEY:1, KEY:-1})
```

(1-Ascending,
-1-Descending)

MongoDB Aggregation

- Aggregations operations process data records and return computed results. Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result. In sql `count(*)` and `with group by` is an equivalent of mongodb aggregation.
- `>db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)`

```
db.train.aggregate([{$group : { _id : "$id", total : { $sum : "$fare" }}}])
```

Stage Expression Accumulator

Aggregate Functions

- `> db.mycol.aggregate(
 [{$group : {_id : "$subject",
 num_count : {$sum : 1}
 }
 }]
)`
- Select subject,count(*) from teachers group by subject
- E.g.
- `db.teachers.aggregate([{$group : {_id: "$address.city", total : {$sum : 1}}])`

Aggregation command expressions

Expression	Description
\$sum	Sums up the defined value from all documents in the collection.
\$avg	Calculates the average of all given values from all documents in the collection.
\$min	Gets the minimum of the corresponding values from all documents in the collection.
\$max	Gets the maximum of the corresponding values from all documents in the collection.
\$push	Inserts the value to an array in the resulting document.
\$addToSet	Inserts the value to an array in the resulting document but does not create duplicates.
\$first	Gets the first document from the source documents according to the grouping. Typically this makes only sense together with some previously applied “\$sort”-stage.
\$last	Gets the last document from the source documents according to the grouping. Typically this makes only sense together with some previously applied “\$sort”-stage.

Aggregate Functions

- \$sum – sums defined value
 - `db.mycol.aggregate([{$group : {_id : "$column", totLikes : {$sum : "$likes"}}}])`
- \$avg – avg of all given values
 - `db.mycol.aggregate([{$group : {_id : "$column", avgLikes : {$avg : "$likes"}}}])`
- \$min - Returns the minimum value from the numeric field
- \$max – Get the maximum value from the numeric field.
- \$first – Gets only the first document from the grouped ones, usually for sorting
- \$last – Returns the last document.
- \$multiply – multiplies fields supplied in array
- \$divide – operator
- \$push – inserts the field value into the result field
- \$addToSet - – Inserts a value into an array of the resulting document, but does not create duplicates

Pipeline Concept

- In UNIX command shell pipeline means the possibility to execute an operation on some input and use the output as the input for the next command and so on. MongoDB also support same concept in aggregation framework. There is a set of possible stages and each of those is taken a set of documents as an input and is producing a resulting set of documents (or the final resulting JSON document at the end of the pipeline).
- This can then in turn again be used for the next stage an so on.
- Aggregation pipelines can have one or more "stages".
- The order of these stages are important. Each stage acts upon the results of the previous stage.

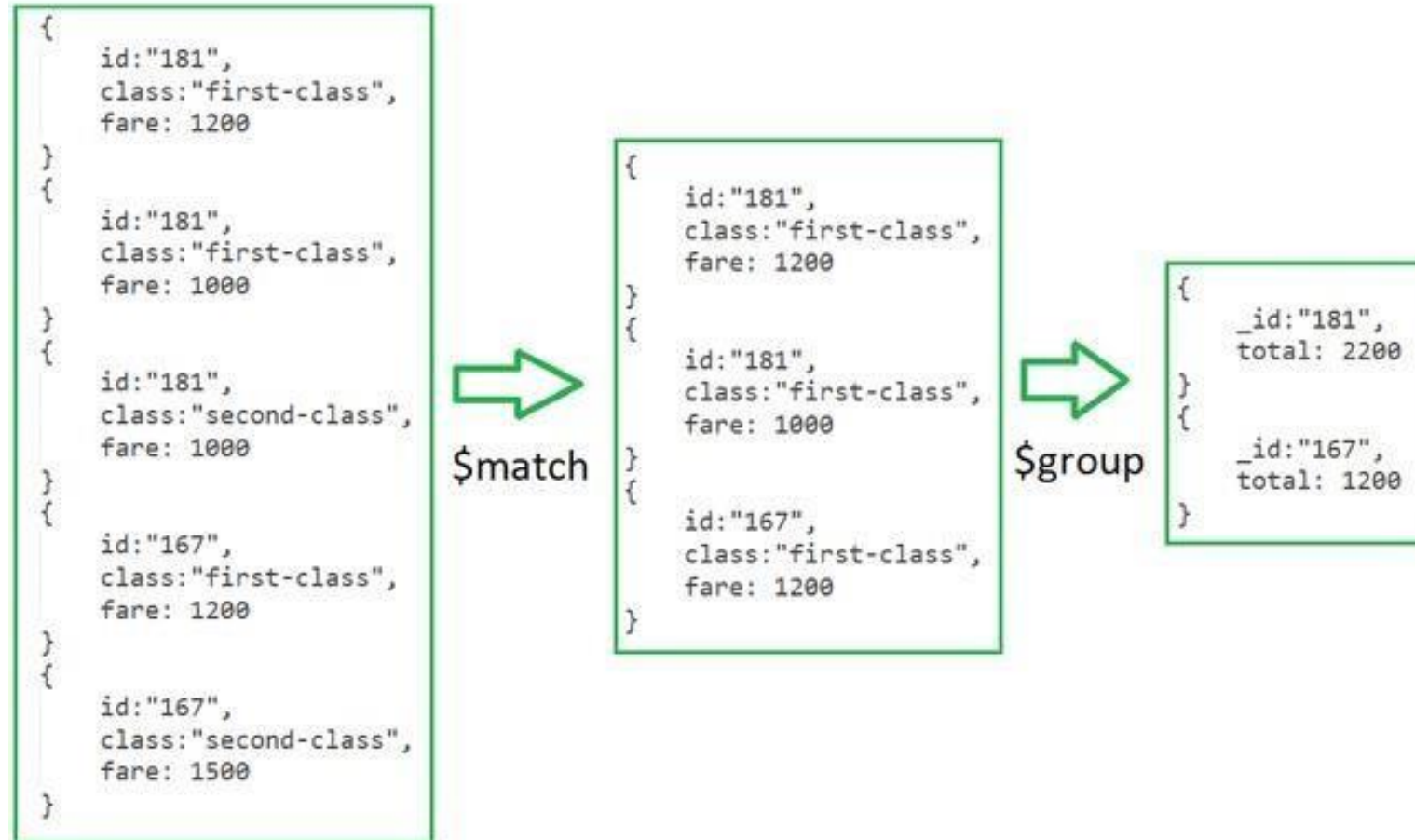
Possible stages in aggregation framework are following:

1. **\$project: Used to select some specific fields from a collection**
2. **\$match:** This is a filtering operation and thus this can reduce the amount of documents that are given as input to the next stage.
3. **\$group:** This does the actual aggregation as discussed above.
4. **\$sort:** Sorts the documents.
5. **\$skip:** With this it is possible to skip forward in the list of documents for a given amount of documents.
6. **\$limit:** This limits the amount of documents to look at by the given number starting from the current position
7. **\$out: it allows you to carry the results of your aggregation over into a new collection, or into an existing one after dropping it**

Pipeline

```
db.train.aggregate( [  
  {$match:{class:"first-class"}},  
  {$group:{_id:"id",total:{$sum:"$fare"}}} ] )
```

} pipeline stages



Pipeline all example

```
db.collection.aggregate([  
  { "$match": { "$text": { "$search": "cake tea" } } },  
  { "$group": { "_id": "field_name", summary : { $sum : 1 } } },  
  { "$sort": { "score": 1/-1 } },  
  { "$limit": limit },  
  { "$skip": skip }  
])
```

Pipeline all example

```
db.collection.aggregate([
  { $project : { "name": 1, "dob" : 0 } },
  { "$match": { "$text": { "$search": "cake tea" } } },
  {"$group": { _id : "field_name", summary : { $sum : 1 } } },
  { "$sort": { "score": 1/-1 } },
  { "$limit": limit },
  { "$skip": skip },
  { "$out" : 'collection name' }
])
```